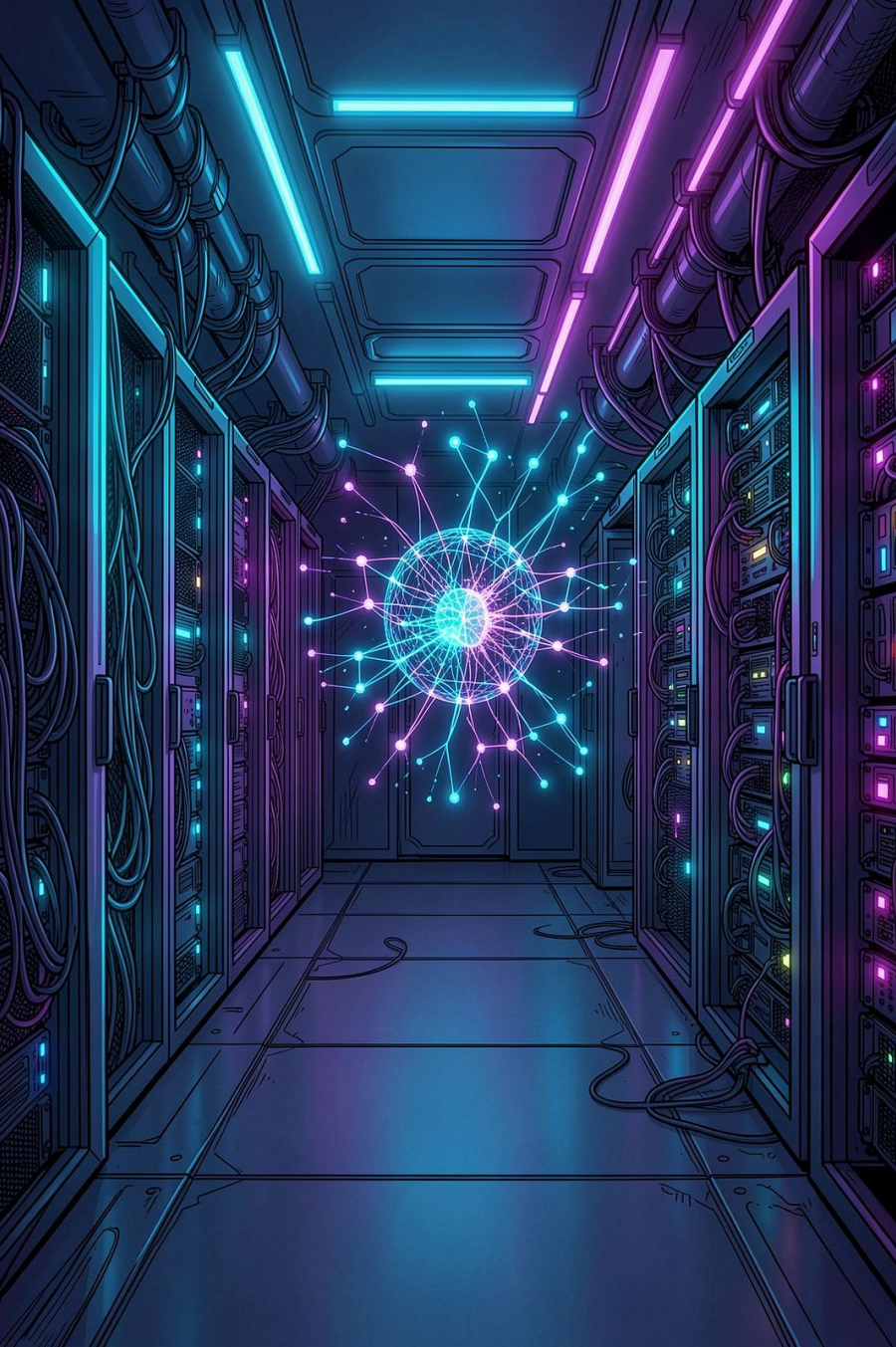




Sistema de Evaluación Automatizada con IA

Defensa oral del proyecto — Pseudocódigo del sistema construido en **Go**
con integración a la API de Groq

GO 1.21

GROQ API

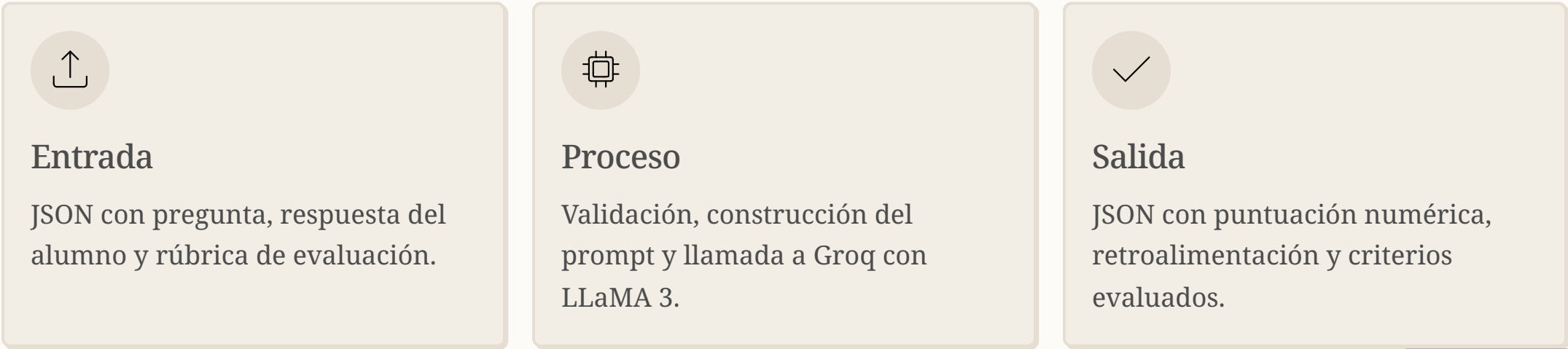
REST

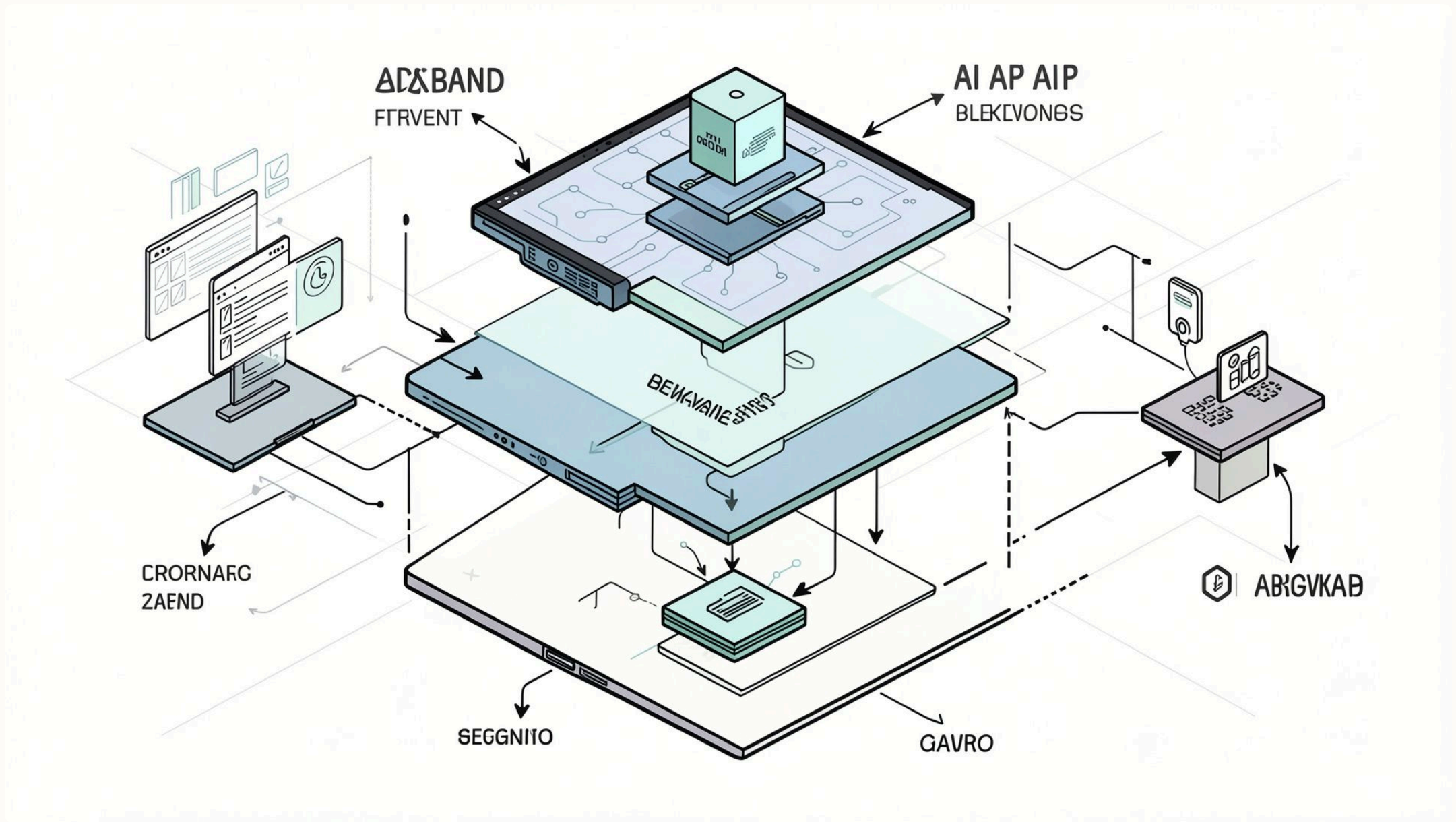
JSON



Objetivo del Sistema

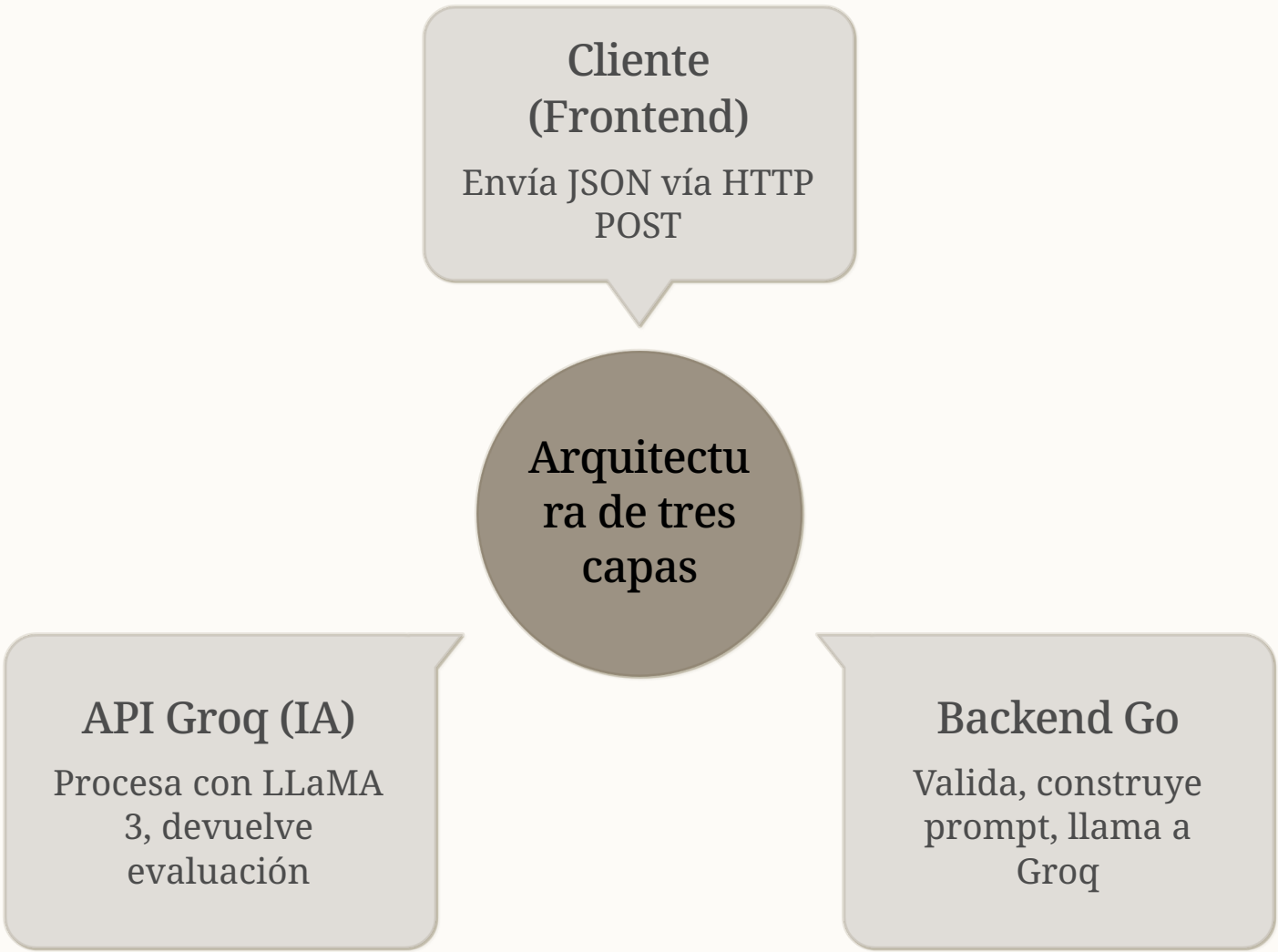
Automatiza la evaluación de respuestas mediante IA generativa, eliminando la revisión manual y devolviendo resultados estructurados en tiempo real.





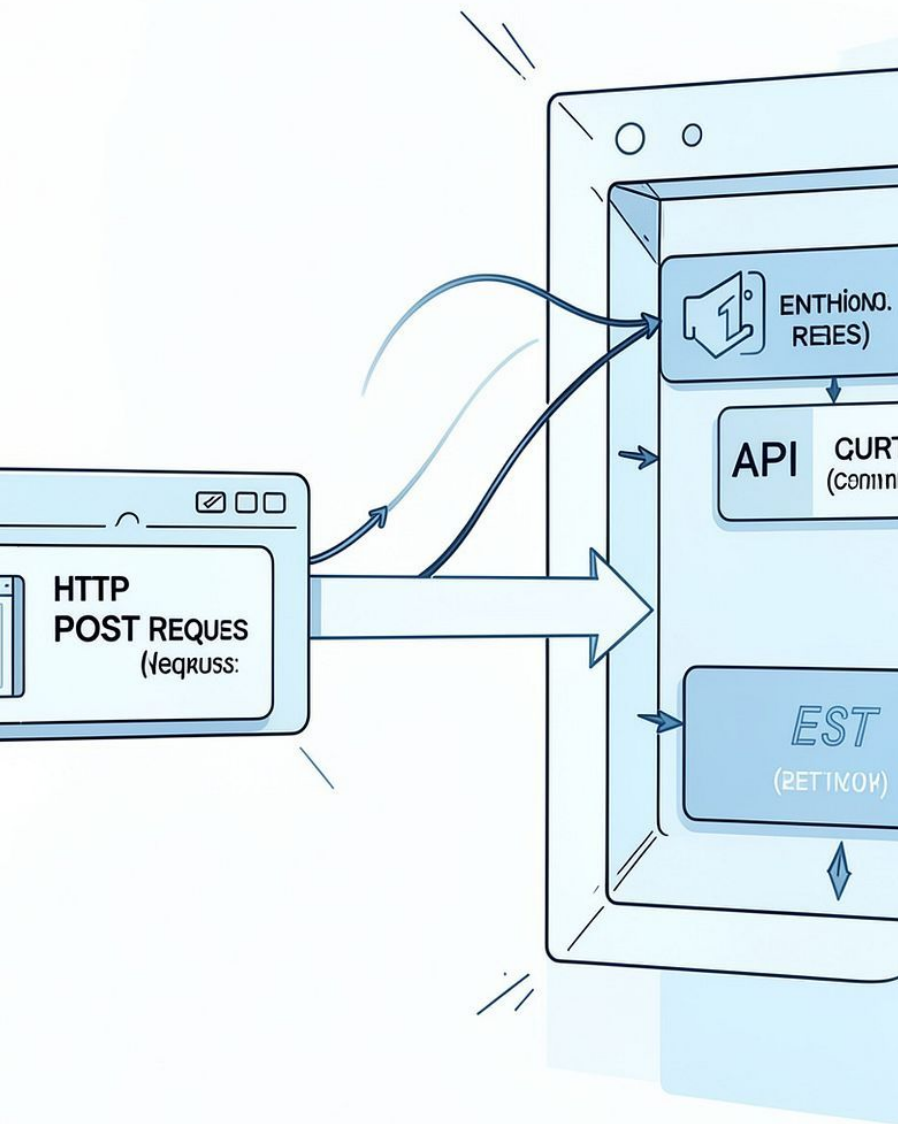
Arquitectura General

El sistema sigue un modelo **cliente-servidor de tres capas** en Go. El backend actúa como intermediario seguro: valida entradas, gestiona la autenticación con Groq y transforma respuestas crudas en datos estructurados para el cliente.



Flujo Principal — POST

/api/evaluar



Pseudocódigo del handler

```
func evaluarHandler(w  
http.ResponseWriter, r  
*http.Request) {  
    // 1. Leer y decodificar cuerpo  
    JSON  
    req := leerBody(r)  
  
    // 2. Validar campos obligatorios  
    si !validar(req) entonces  
        responderError(w, 400)  
        retornar  
  
    // 3. Ejecutar evaluación con IA  
    resultado := evaluarConIA(req)  
  
    // 4. Responder JSON al cliente  
    responderJSON(w, 200,  
    resultado)  
}
```

Puntos clave

- **Decodificación segura:** `json.NewDecoder` con manejo de errores.
- **Validación temprana:** campos vacíos o mal formados se rechazan antes de llamar a la API.
- **Separación de responsabilidades:** el handler delega lógica a funciones auxiliares.
- **Respuesta uniforme:** siempre JSON, incluso en errores.

Construcción de la Lógica Central

Pseudocódigo —
evaluarConIA()

```
func evaluarConIA(req Request)
Resultado {
  // 1. Construir prompt con
  contexto
  prompt :=
  armarPrompt(req.pregunta,
               req.respuesta,
               req.rubrica)

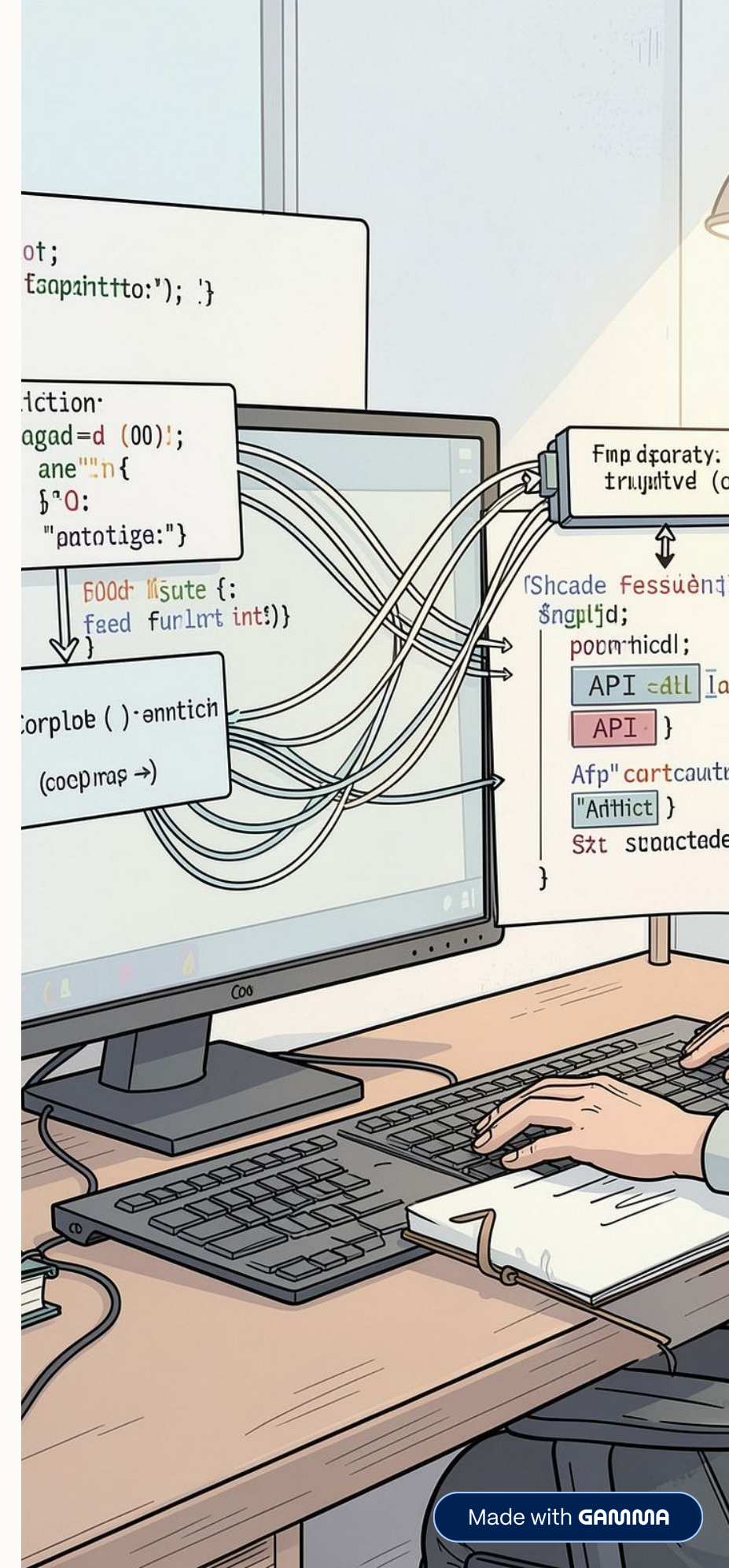
  // 2. Llamar a API externa
  raw := llamarGroq(prompt)

  // 3. Parsear respuesta cruda
  parsed := extraerDatos(raw)

  // 4. Retornar estructura tipada
  retornar Resultado{
    puntaje: parsed.score,
    feedback: parsed.texto
  }
}
```

Estrategia de diseño

- **Prompt estructurado:** se inyectan pregunta, respuesta y rúbrica en un template claro.
- **Función pura:** sin efectos secundarios, fácil de testear con mocks.
- **Transformación explícita:** la respuesta cruda se convierte a estructura tipada antes de retornar.



Comunicación con la API Groq

Pseudocódigo — llamarGroq()

```
func llamarGroq(prompt string)
string {
    // 1. Armar payload JSON
    payload := {
        model: "llama3-70b-8192",
        messages: [{role: "user",
            content: prompt}]
    }

    // 2. Configurar headers
    headers := {
        "Authorization": "Bearer " +
API_KEY,
        "Content-Type":
"application/json"
    }

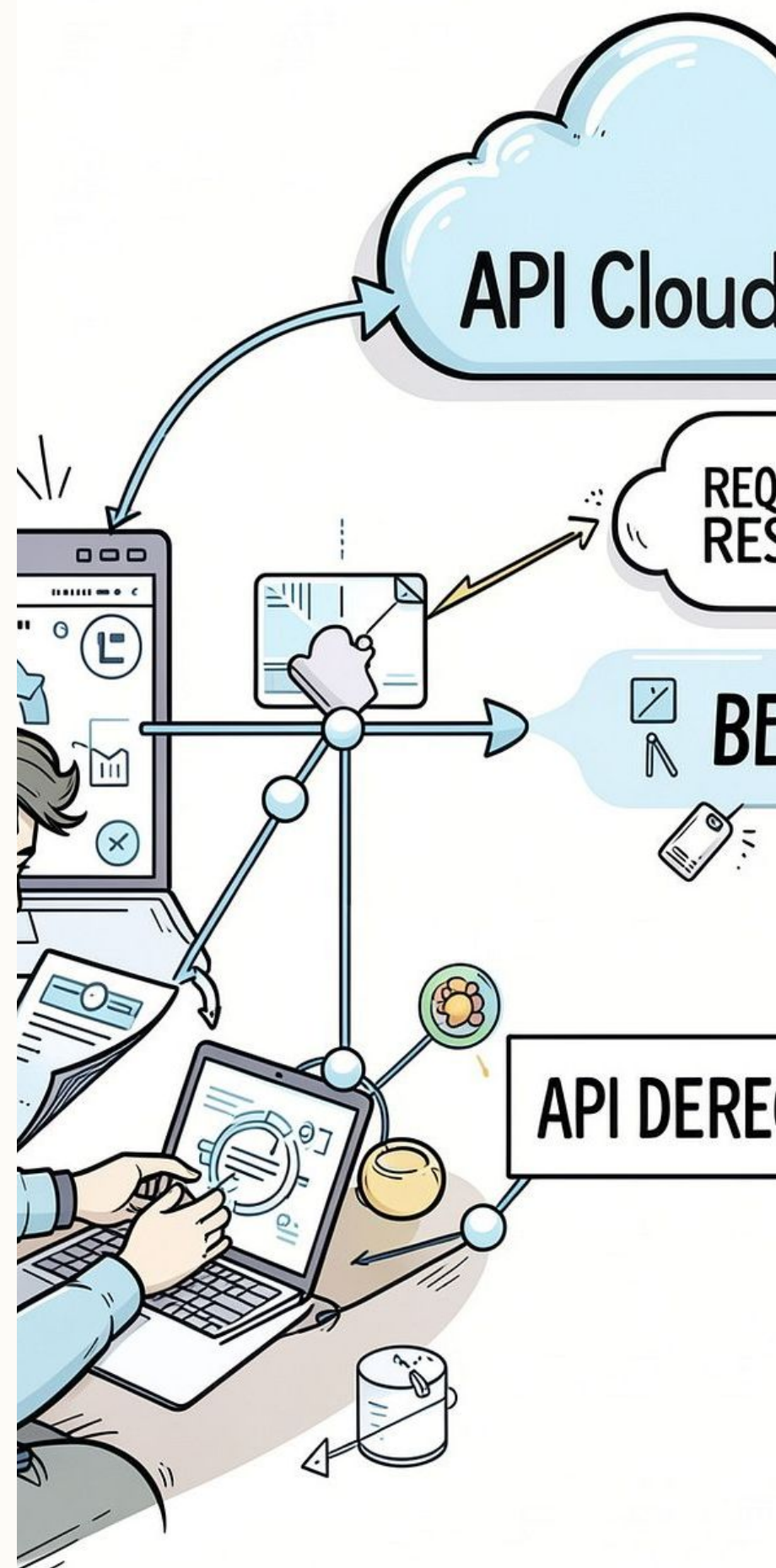
    // 3. Enviar POST a Groq
    resp, err := http.Post(url,
headers, payload)

    // 4. Manejar errores por código
    si resp.StatusCode != 200
    entonces
        manejarError(resp.StatusCode)
    si err != nil entonces
        panic(err)

    retornar resp.Body
}
```

Gestión de errores

- **401 Unauthorized:** API key inválida o expirada.
- **429 Too Many Requests:** rate limit alcanzado — reintento con backoff.
- **5xx Server Error:** error del proveedor — registrar y notificar.
- **Timeout de red:** contexto con `context.WithTimeout` de 30 segundos.



Procesamiento de la Respuesta

Pseudocódigo — extraerDatos()

```
func extraerDatos(raw string)
DatosParseados {
    // 1. Deserializar JSON de Groq
    var resp GroqResponse
    json.Unmarshal(raw, &resp)

    // 2. Extraer contenido del
    mensaje
    contenido :=
    resp.Choices[0].Message.Content

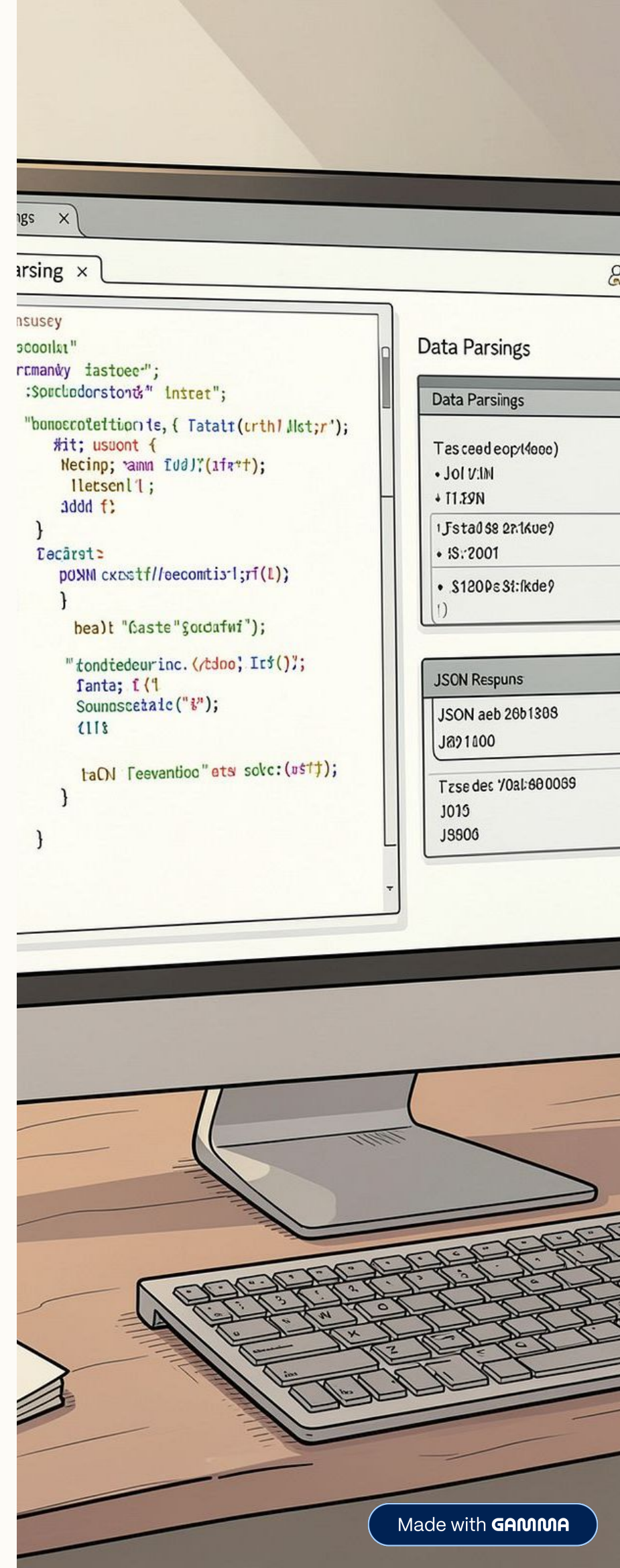
    // 3. Buscar patrón de
    puntuación
    // Ej: "Puntuación: 8/10"
    score :=
    regexBuscar(`Puntuación: (\d+)/`,
        contenido)

    // 4. Extraer feedback (resto del
    texto)
    feedback :=
    limpiarTexto(contenido)

    retornar DatosParseados{
        score: score,
        texto: feedback
    }
}
```

Estrategia de extracción

- **JSON nativo:** se usa `encoding/json` de Go para la capa externa.
- **Regex focalizado:** patrón simple para extraer la puntuación numérica del texto libre.
- **Limpieza defensiva:** se eliminan saltos de línea y espacios extra antes de retornar.
- **Fallback seguro:** si el regex no coincide, se asigna `score: 0` y se registra un warning.



Flujo End-to-End

El proceso completo —desde la recepción del request hasta la entrega del resultado— se ejecuta en menos de 3 segundos en condiciones normales de red, con tolerancia a fallos en cada etapa crítica.

01

Recepción

Cliente envía JSON con pregunta, respuesta y rúbrica

02

Validación

Backend verifica campos y formato en Go

03

Evaluación IA

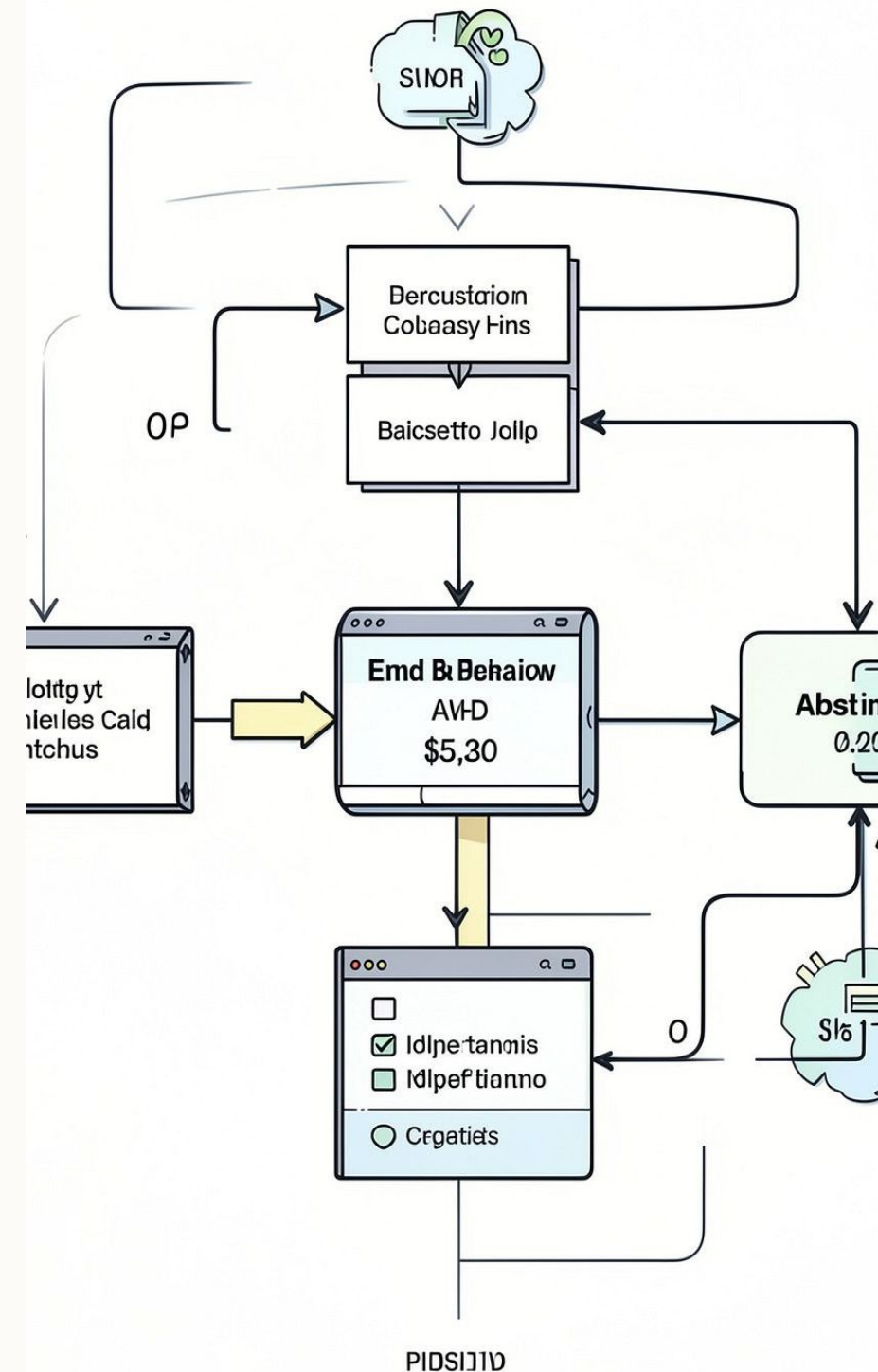
Se construye prompt y se llama a Groq API

04

Respuesta

Se parsea, estructura y devuelve JSON al cliente

-OLFIEND

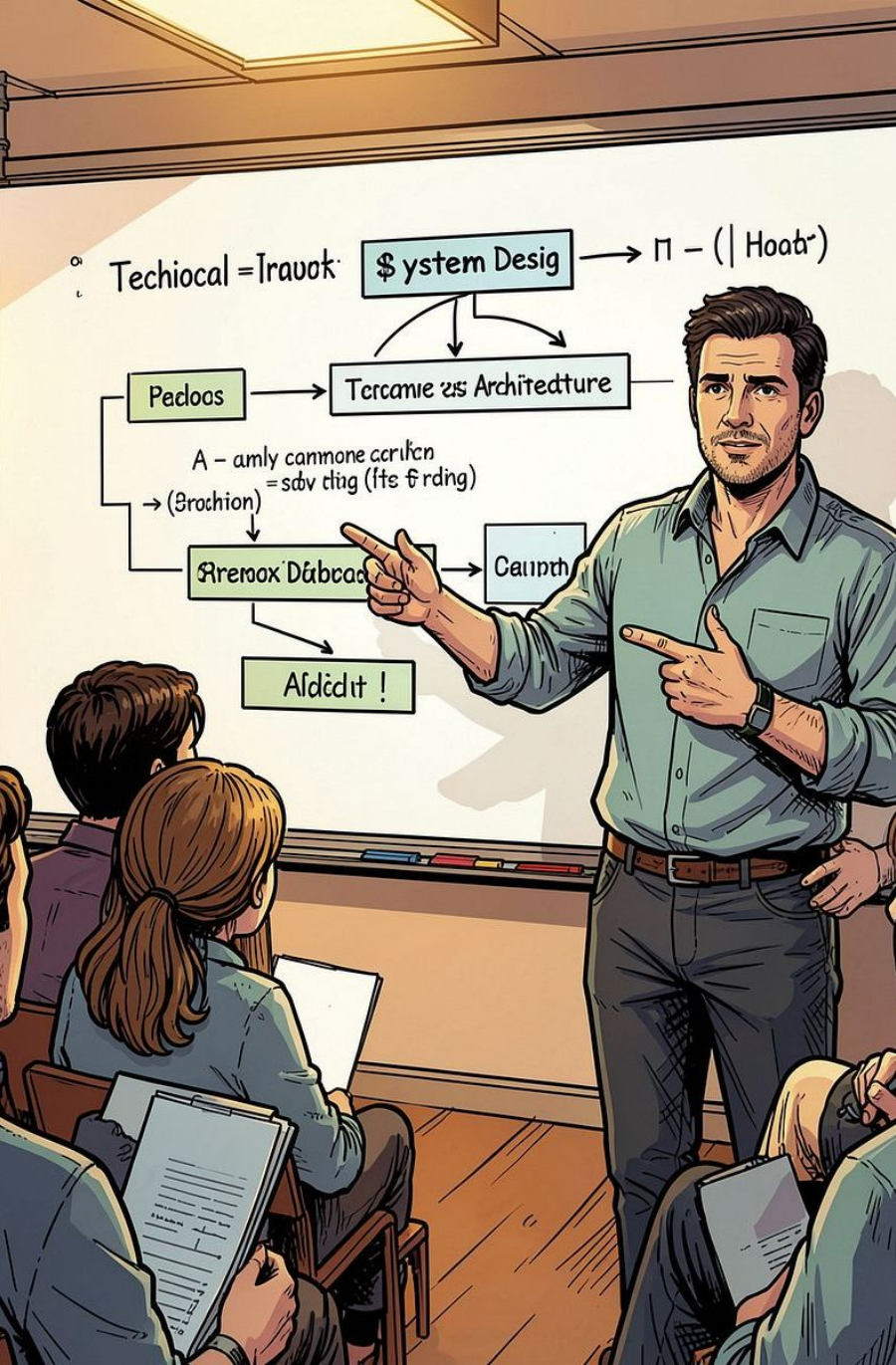


Criterios de Evaluación del Sistema

Criterio	Nivel A — Excelente	Nivel B — Satisfactori o	Nivel C — En desarrollo
Validación de entrada	Campos, tipos y límites verificados	Solo campos obligatorios	Validación mínima o ausente
Gestión de errores API	Códigos 4xx/5xx con reintento	Manejo básico de errores	Sin manejo explícito
Estructura del prompt	Contexto, rúbrica y formato claros	Prompt funcional sin contexto	Prompt ambiguo o incompleto
Parseo de respuesta	Regex + JSON con fallback seguro	Extracción parcial	Sin parseo estructurado



Conclusiones y Aprendizajes



Arquitectura por capas

Separar handler, lógica de negocio y comunicación externa redujo la complejidad y facilitó los tests unitarios con mocks de la API.

Diseño del prompt como código

Tratar el prompt como una plantilla estructurada —no como texto libre— mejoró la consistencia de las evaluaciones y simplificó el parseo.

Go + IA: combinación efectiva

La concurrencia nativa de Go y su tipado fuerte son ideales para orquestar llamadas a APIs externas con control preciso de errores y tiempos de respuesta.